Flask Cookies & Sessions

Classroom Notes with Step-by-Step Practical

Goal: Understand why cookies and sessions are used, how Flask creates/reads/deletes them, and how session state works across requests.

1. Start the Class: Why Do We Need Them?

HTTP is stateless. By default, one request does not automatically remember another request. Ask students: "When we move from one page to another, how does a website remember us?" Cookies and sessions help maintain information across requests.

```
Request 1 -> Server
Request 2 -> Server
Request 3 -> Server
```

2. What is a Cookie?

A cookie is a small piece of data that a website asks the browser to store. Examples include username, language preference, theme, or other small preferences.

```
username = Deepak
language = English
theme = dark
```

3. Create the Basic Flask App

First create a simple Flask application. Run it before adding cookies so students

understand the starting point.

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Cookie and Session Class"

if __name__ == "__main__":
    app.run(debug=True)
```

4. Set a Cookie

To set a cookie, create a response using make_response(), then attach a cookie using set_cookie().

```
from flask import Flask, make_response

@app.route("/set-cookie")
def set_cookie():
    response = make_response("Cookie Created")
    response.set_cookie("username", "Deepak")
    return response
```

5. Explain set_cookie()

In set_cookie("username", "Deepak"), username is the cookie name and Deepak is its value. The browser receives the Set-Cookie response header and stores the

cookie for this website.

```
Cookie Name  = username
Cookie Value = Deepak
```

6. Show the Cookie in Browser

Open Chrome DevTools and inspect the cookie. This is important because students can see that the cookie is stored by the browser.

```
F12
-> Application
-> Cookies
-> http://127.0.0.1:5000
```

7. Read a Cookie

Use request.cookies.get() to read a cookie sent by the browser with the current request.

```
from flask import request

@app.route("/get-cookie")
def get_cookie():
    username = request.cookies.get("username")

    if username:
        return f"Welcome {username}"

    return "Cookie Not Found"
```

8. Cookie Flow

The server first asks the browser to store the cookie. On later requests, the browser can send the matching cookie back to the server.

```
/set-cookie
-> Flask sends Set-Cookie
-> Browser stores username=Deepak
-> /get-cookie
-> Browser sends cookie
-> Flask reads it
-> Welcome Deepak
```

9. Delete a Cookie

Use delete_cookie() on the response object to remove the cookie.

```
@app.route("/delete-cookie")
def delete_cookie():
    response = make_response("Cookie Deleted")
    response.delete_cookie("username")
    return response
```

10. Cookie Expiry

max_age controls how long the cookie should remain valid. It is expressed in seconds.

```
# 60 seconds
response.set_cookie("username", "Deepak", max_age=60)
```

```
# 5 minutes
max_age=300

# 1 hour
max_age=3600

# 1 day
max_age=86400
```

11. Complete Cookie Program

Use this program for the final cookie practical in class.

```
from flask import Flask, make_response, request

app = Flask(__name__)

@app.route("/")
def home():
    return "Cookie Example"

@app.route("/set-cookie")
def set_cookie():
    response = make_response("Cookie Created")
    response.set_cookie("username", "Deepak", max_age=3600)
    return response

@app.route("/get-cookie")
def get_cookie():
    username = request.cookies.get("username")
    if username:
        return f"Welcome {username}"
    return "Cookie Not Found"

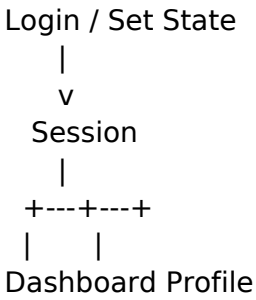
@app.route("/delete-cookie")
```

```
def delete_cookie():
    response = make_response("Cookie Deleted")
    response.delete_cookie("username")
    return response

if __name__ == "__main__":
    app.run(debug=True)
```

12. What is a Session?

A session is a mechanism used to maintain user-specific state across multiple requests. Example: a user enters the application, opens dashboard, profile, or another page, and the application needs to remember some user-specific information.



13. Configure Flask Session

Import session and configure a SECRET_KEY. Flask uses the secret key to cryptographically sign its default session cookie.

```
from flask import Flask, session

app = Flask(__name__)
app.secret_key = "my-secret-key"
```

14. Set Session Data

Flask session behaves similarly to a Python dictionary. Store values using key-value syntax.

```
@app.route("/set-session")
def set_session():
    session["username"] = "Deepak"
    session["email"] = "deepak@gmail.com"
    session["role"] = "Trainer"

    return "Session Created"
```

15. Read Session Data

Use session.get() to safely retrieve a session value.

```
@app.route("/get-session")
def get_session():
    username = session.get("username")

    if username:
        return f"Welcome {username}"

    return "Session Not Found"
```

16. Delete One Session Value

session.pop() removes a single key from the session.

```
@app.route("/delete-session")
def delete_session():
    session.pop("username", None)
    return "Username Removed"
```

17. Clear the Complete Session

session.clear() removes all values stored in the current session. This is commonly used when ending a login session.

```
@app.route("/clear-session")
def clear_session():
    session.clear()
    return "Session Cleared"
```

18. Simple Session Practical

This example demonstrates setting state, protecting a page using session data, reading it on another page, and clearing it.

```
from flask import Flask, session

app = Flask(__name__)
app.secret_key = "my-secret-key"

@app.route("/")
def home():
    return "Session Example"

@app.route("/login")
```

```
def login():
    session["username"] = "Deepak"
    session["role"] = "Trainer"
    return "Session Created"

@app.route("/dashboard")
def dashboard():
    if "username" in session:
        return f"Welcome {session['username']}"
    return "Please Login First"

@app.route("/profile")
def profile():
    username = session.get("username")
    role = session.get("role")

    if username:
        return f"Name: {username}, Role: {role}"

    return "Session Not Found"

@app.route("/logout")
def logout():
    session.clear()
    return "Session Cleared"

if __name__ == "__main__":
    app.run(debug=True)
```

19. Test Session in This Order

Ask students to open the routes in the following order. After clearing the session, opening dashboard again should show

"Please Login First".

/login
/dashboard
/profile
/logout
/dashboard

20. Important Flask Session Point

Flask's default session is not automatically a server-side database session. By default, Flask serializes session data into a cryptographically signed client-side cookie. The signature protects integrity, but the data is not meant to be treated as secret encrypted storage. Never store passwords or other sensitive secrets in the session.

Good:
session["user_id"] = 101

Avoid:
session["password"] = "12345"

21. Cookie vs Session

Cookie	Session
Browser stores cookie data	Flask session API maintains user state
set_cookie()	session["key"] = value

Cookie	Session
request.cookies.get()	session.get()
delete_cookie()	session.pop() / session.clear()
Useful for small preferences	Useful for user-specific state

22. Student Practice

1. Create a cookie named city with value Noida.

2. Create /show-city and display the city cookie.

3. Make the cookie expire after 5 minutes.

4. Store name, email and course in a session.

5. Create /student that opens only when name exists in the session.

6. Create /logout that clears the complete session.